// C++ implementation of search and insert operations on Trie DS

```
#include <bits/stdc++.h>
using namespace std;
const int ALPHABET_SIZE = 26;

struct TrieNode
{
   struct TrieNode *children[ALPHABET_SIZE];
   bool isEndOfWord;
};

struct TrieNode *getNode(void)
{
   struct TrieNode *pNode =  new TrieNode;
   pNode->isEndOfWord = false;

   for (int i = 0; i < ALPHABET_SIZE; i++)
      pNode->children[i] = NULL;

   return pNode;
}

// (If not present, inserts key into trie) vs (If the key is prefix of trie node, just marks leaf node)
void insert(struct TrieNode *root, string key)
{
   struct TrieNode *pCrawl = root;

   for (int i = 0; i < key.length(); i++)
   {
      int index = key[i] - 'a';
      if (!pCrawl->children[index])
         pCrawl->children[index] = getNode();

      pCrawl = pCrawl->children[index];
   }

   // mark last node as leaf
   pCrawl->isEndOfWord = true;
}



// Returns true if key presents in trie, else false
bool search(struct TrieNode *root, string key)
{
   struct TrieNode *pCrawl = root;

   for (int i = 0; i < key.length(); i++)
```

```
    {
        int index = key[i] - 'a';
        if (!pCrawl->children[index])
            return false;

        pCrawl = pCrawl->children[index];
    }

    return (pCrawl != NULL && pCrawl->isEndOfWord);
}

int main()
{
    // Input keys (use only 'a' through 'z' and lower case)
    string keys[] = {"the", "a", "there",  "answer", "any", "by",  "bye", "their" };
    int n = sizeof(keys)/sizeof(keys[0]);

    struct TrieNode *root = getNode();

    // Construct Trie DS and insert to Trie
    for (int i = 0; i < n; i++)
        insert(root, keys[i]);

    // Search for different keys
    search(root, "the")? cout << "Yes\n" :
                         cout << "No\n";
    search(root, "these")? cout << "Yes\n" :
                          cout << "No\n";
    return 0;
}
```